

LAPORAN PRAKTIKUM
PRAKTIKUM PENGUJIAN PERANGKAT LUNAK
PERTEMUAN 9
ADVANCED INTERACTION AND WAIT STRATEGY



Disusun oleh:

Nama : Januarsyah Akbar
NIM : 24/535846/SV/24314
Kelas : PL4B2
Dosen Pengampu : Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA

May 5, 2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Hasil dan Pembahasan	4
2.1 Latihan (Advanced Interactions)	4
2.1.1 Metode Pengujian <code>testHoverShowsName</code>	4
2.1.2 Metode Pengujian <code>testKeyPressesShift</code>	5
2.2 Tugas (Wait Strategies)	7
2.2.1 Metode Pengujian <code>testCase1_noSuchElementAndWaits</code>	7
2.2.2 Metode Pengujian <code>testCase5_staleElement</code>	8
Kesimpulan	10

Tujuan Praktikum

1. Mahasiswa mampu memahami dan mengimplementasi *Action Class* untuk melakukan interaksi situs yang kompleks seperti *hover*, *keypresses*, dan *drag and drop*.
2. Mahasiswa mampu menggunakan dan memahami konsep mekanisme tunggu (*Wait Strategy*) seperti penggunaan *Implicit Wait*, *Explicit Wait*, beserta penanganan terhadap pelbagai *Exceptions*.

Hasil dan Pembahasan

2.1 Latihan (Advanced Interactions)

Pada bagian latihan ini, kita akan mempraktikkan kelas `Actions` dari Selenium WebDriver untuk menangani interaksi pengguna tingkat lanjut (*Advanced Interaction*) yang tidak dapat ditangani dengan utilitas `WebElement` standar. Interaksi yang diujikan mencakup *hover* ke elemen tertentu serta simulasi ketukan *keyboard* (tombol *SHIFT*).

Pengujian ini mengimplementasikan skenario interaksi yang kompleks dari halaman latihan "The Internet Herokuapp" yang dibagi ke dalam dua buah metode pengujian berbeda.

2.1.1 Metode Pengujian `testHoverShowsName`

```
1  @Test
2  @DisplayName("Hover: nama user1 muncul saat hover gambar pertama")
3  public void testHoverShowsName() {
4      driver.get("https://the-internet.herokuapp.com/hovers");
5
6      WebElement firstFigure = driver.findElement(By.cssSelector(".figure:nth-of-
↪ type(1)"));
7      actions.moveToElement(firstFigure).perform();
8
9      WebElement caption = firstFigure.findElement(By.cssSelector(".figcaption h5"
↪ ));
10     String text = caption.getText().trim();
11     Assertions.assertTrue(text.toLowerCase().contains("user1"), "Caption harus
↪ mengandung 'user1' tetapi ditemukan: " + text);
12 }
```

Listing .1: Metode Utuh `testHoverShowsName`

Pengujian ini memverifikasi interaksi *hover* pada gambar untuk memunculkan profil sekunder (caption).

1. Navigasi dan Pencarian Target

```
1  driver.get("https://the-internet.herokuapp.com/hovers");
2  WebElement firstFigure = driver.findElement(By.cssSelector(".figure:nth-of-type
↪ (1)"));
```

Pada tahap ini, *webdriver* diarahkan ke tautan tujuan utama. Selanjutnya, program mencari komponen membungkus struktur gambar pertama dengan memanfaatkan *CSS Selector*.

2. Simulasi Tindakan Hover

```
1 actions.moveToElement(firstFigure).perform();
```

Kelas `Actions` mengambil alih pergerakan virtual dengan menautkan koordinat *pointer* tetikus (*mouse*) ke arah elemen yang telah dideteksi sebelumnya, disusul pemanggilan method `.perform()` untuk mengeksekusi secara nyata interaksi tersebut.

3. Verifikasi Pemuatan Teks (Assertion)

```
1 WebElement caption = firstFigure.findElement(By.cssSelector(".figcaption h5"));
2 String text = caption.getText().trim();
3 Assertions.assertTrue(text.toLowerCase().contains("user1"), "Caption harus
↪ mengandung 'user1' tetapi ditemukan: " + text);
```

Setelah elemen di-*hover*, teks identitas bertuliskan "name: user1" baru dirender menjadi visibel oleh browser. Langkah pamungkasnya adalah memancing propeerti teks tersebut, mengkonversinya (`toLowerCase()`), lalu memastikannya lolos persyaratan pengujian `Assertions`.

2.1.2 Metode Pengujian `testKeyPressesShift`

```
1 @Test
2 @DisplayName("Key presses: tekan SHIFT dan verifikasi teks hasil")
3 public void testKeyPressesShift() {
4     driver.get("https://the-internet.herokuapp.com/key_presses");
5
6     WebElement input = driver.findElement(By.id("target"));
7     actions.click(input).sendKeys(Keys.SHIFT).perform();
8
9     WebElement result = driver.findElement(By.id("result"));
10    String text = result.getText().trim();
11    Assertions.assertTrue(text.toUpperCase().contains("SHIFT"), "Hasil harus
↪ mengandung SHIFT tetapi ditemukan: " + text);
12 }
```

Listing .2: Metode Utuh `testKeyPressesShift`

Pengujian ini memvalidasi apakah pengiriman sinyal tombol spesifik dari *keyboard* pengguna (dalam hal ini *SHIFT*) ditangkap dan dihimpun oleh *event listener* DOM laman eksternal.

1. Inisialisasi Laman dan Input Keyboard

```
1 driver.get("https://the-internet.herokuapp.com/key_presses");
2 WebElement input = driver.findElement(By.id("target"));
3 actions.click(input).sendKeys(Keys.SHIFT).perform();
```

Pertama-tama agen mengakses pranala penyedia tes ketukan tombol. Aktivasi ketukan otomatis dilakukan berjejer dengan mengincar fungsional komponen target (`click()`) dan menyuntikkan instruksi mekanis menekan tombol `SHIFT` ke dalamnya menggunakan abstraksi `Keys.SHIFT`.

2. Validasi Keluaran

```
1   WebElement result = driver.findElement(By.id("result"));
2   String text = result.getText().trim();
3   Assertions.assertTrue(text.toUpperCase().contains("SHIFT"), "Hasil harus
↪ mengandung SHIFT tetapi ditemukan: " + text);
```

Sistem situs dari pihak *heroku* akan mem-parsing keluaran hasil simulasi secara seketika pada sebuah komponen teks bernaung `#result`. Skrip mengakhiri sesi dengan melintasi DOM elemen spesifik tersebut dan melakukan asersi kecocokan untai barisan penulisan ("SHIFT").

2.2 Tugas (Wait Strategies)

Pada tahap pengerjaan tugas, kita berfokus pada berbagai ragam strategi penyesuaian waktu asinkronisasi (*wait strategies*) dalam Selenium untuk menangkalkan fenomena kegagalan (*flaky test*) pada elemen web dinamis. Mekanisme pengamatan dilakukan pada situs "Practice Test Exceptions". Objek penaksirannya melibatkan validasi terhadap kelas-kelas turunan *Exception* bila kondisi-kondisi tunggu seperti `Thread.sleep()`, *implicit wait*, dan *explicit wait* dipicu secara terisolasi.

2.2.1 Metode Pengujian testCase1_noSuchElementExceptionAndWaits

```
1  @Test
2  @DisplayName("Test Case 1 - demonstrasi NoSuchElementException dan strategi wait
↪ ")
3  public void testCase1_noSuchElementExceptionAndWaits() {
4      driver.get("https://practicetestautomation.com/practice-test-exceptions/");
5
6      By missing = By.id("this-id-does-not-exist-hopefully");
7
8      // 1) Tanpa wait/implicit
9      driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(0));
10     Assertions.assertThrows(NoSuchElementException.class, () -> driver.
↪ findElement(missing), "Harus melempar NoSuchElementException");
11
12     // 2) Dengan implicit wait
13     driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));
14     long start = System.nanoTime();
15     Assertions.assertThrows(NoSuchElementException.class, () -> driver.
↪ findElement(missing));
16
17     // 3) Dengan explicit wait
18     WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(3));
19     Assertions.assertThrows(org.openqa.selenium.TimeoutException.class, () ->
↪ wait.until(ExpectedConditions.presenceOfElementLocated(missing)));
20
21     // 4) Thread.sleep
22     try {
23         Thread.sleep(2000);
24     } catch (InterruptedException e) {
25         Thread.currentThread().interrupt();
26     }
27     Assertions.assertThrows(NoSuchElementException.class, () -> driver.
↪ findElement(missing));
28 }
```

Listing .3: Metode Utuh testCase1_noSuchElementExceptionAndWaits

Fungsi ini ditujukan untuk membedakan sifat perilaku berbagai algoritma waktu jeda tatkala sebuah elemen yang dimintakan tidak pernah bermukim di dalam DOM.

1. Tanpa Sistem Tunggu (Implicit Wait = 0)

```
1  By missing = By.id("this-id-does-not-exist-hopefully");
2  driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(0));
3  Assertions.assertThrows(NoSuchElementException.class, () -> driver.findElement(
↪ missing), "Harus melempar NoSuchElementException");
```

Saat *property latency* driver diatur sebesar 0, eksekusi pemanggilan pemetaan tidak akan menoleransi kegagalan dan langsung memvonis sistem dengan pencetakan `NoSuchElementException`.

2. Skenario Implicit Wait (5 Detik)

```
1 driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));
2 long start = System.nanoTime();
3 Assertions.assertThrows(NoSuchElementException.class, () -> driver.findElement(
  ↪ missing));
```

Pendekatan ini menyisipkan durasi kompromi di hulu (*global driver*). Jika *element* tak terlihat, *driver* akan memantau ulang *source target* terus-menerus selama 5 detik hingga tiba di masa habisnya periode lalu melempar turunan *Exception* yang sama.

3. Skenario Explicit Wait (Kondisional)

```
1 WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(3));
2 Assertions.assertThrows(org.openqa.selenium.TimeoutException.class, () -> wait.
  ↪ until(ExpectedConditions.presenceOfElementLocated(missing)));
```

Modul eksplisit dioperasikan eksklusif terhadap sebuah entitas alih-alih seluruh sesi. Jika `ExpectedConditions` dalam batas waktu (3 detik) urung dicapai, kerangka Selenium otomatis memutuskan *event loop* dan meletuskan `TimeoutException`.

4. Interupsi Konvensional (Thread.sleep)

```
1 try {
2     Thread.sleep(2000);
3 } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
4 Assertions.assertThrows(NoSuchElementException.class, () -> driver.findElement(
  ↪ missing));
```

Memanfaatkan fungsional Java untuk mematikan komputasi secara instan (*hard delay*) selama 2 detik tak berkorelasi terhadap stabilitas siklus elemen, sehingga error tetap merujuk secara statis ke dalam `NoSuchElementException`.

2.2.2 Metode Pengujian `testCase5_staleElement`

```
1 @Test
2 @DisplayName("Test Case 5 - demonstrasi StaleElementReferenceException dan
  ↪ solusinya")
3 public void testCase5_staleElement() {
4     driver.get("https://practicetestautomation.com/practice-test-exceptions/");
5
6     WebElement heading = driver.findElement(By.cssSelector("h1, h2, h3"));
7     String text = heading.getText();
8     Assertions.assertNotNull(text);
9
10    driver.navigate().refresh();
11
12    Assertions.assertThrows(StaleElementReferenceException.class, () -> heading.
  ↪ getText());
13
```

```

14     WebElement headingNew = driver.findElement(By.cssSelector("h1, h2, h3"));
15     Assertions.assertNotNull(headingNew.getText());
16 }

```

Listing .4: Metode Uguh `testCase5_staleElement`

Sesi blok metodologi di bawah ini dipertunjukkan untuk merepresentasikan dan menangani skema kehilangan tautan (*losing bindings*) pada *reference object* antar eksekusi perenderan skrip.

1. Penetapan Elemen Laman Awal

```

1     WebElement heading = driver.findElement(By.cssSelector("h1, h2, h3"));
2     String text = heading.getText();
3     Assertions.assertNotNull(text);

```

Elemen primer pertama kali ditinjau dari posisi absolutnya di awal muatan DOM berjalan mulus berujung pada eksistensi objek `text` yang tak bernilai null.

2. Refleksi Peremajaan Relasi (Stale Error)

```

1     driver.navigate().refresh();
2     Assertions.assertThrows(StaleElementReferenceException.class, () -> heading.
↪     getText());

```

Pemuatan kompilasi ulang (termasuk manipulasi aset maupun *refresh()*) menghancurkan relasi historis indeks, mengucilkan *instance object* walau rupa DOM secara visual terbilang identik. Akibatnya eksekusi ulang meledakkan `StaleElementReferenceException`.

3. Mekanisme *Re-locate* Pemulihan Memori

```

1     WebElement headingNew = driver.findElement(By.cssSelector("h1, h2, h3"));
2     Assertions.assertNotNull(headingNew.getText());

```

Cara baku menghempaskan masalah elemen usang / kadaluwarsa melingkupi pendaftaran pencarian pengait baru dan pembentukan variabel pengganti sesuai siklus perenderan web termutakhir.

Repository

akses kode lengkap untuk latihan dan tugas praktikum ini dapat ditemukan pada repository GitHub berikut:

GitHub Repository

Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan kakas bantu seperti Selenium WebDriver memberikan fungsionalitas lanjutan yang ekstensif melalui kelas *Actions* untuk menangani perilaku-perilaku non-trivial antarmuka web, semisal interaksi *hover*, *keypress*, maupun *drag-and-drop*. Implementasi tersebut mendemonstrasikan keunggulan *automated testing* yang sangat mirip dengan pola natural pengguna manusia.

Lebih lanjut, eksperimentasi seputar *Wait Strategy* yang diujikan turut mevalidasi esensi mekanisme pengkondisian siklus waktu dalam menangkal kerentanan terhadap stabilitas dan varians pemuatan aset serta *render delay*. Eksekusi penanganan menggunakan opsi *Implicit Wait* maupun *Explicit Wait* jauh lebih aman serta efisien bila disandingkan dengan fungsi *blocking* mutlak turunan bawaan Java seperti `Thread.sleep()`, mereduksi latensi dan waktu uji, seraya memungkinkan penyusunan skenario *Exception handling* logis yang terukur semisal pada kasus `NoSuchElementException` dan `StaleElementReferenceException`.